

CNNs

Prof. CV Jawahar

IIIT-Hyderabad

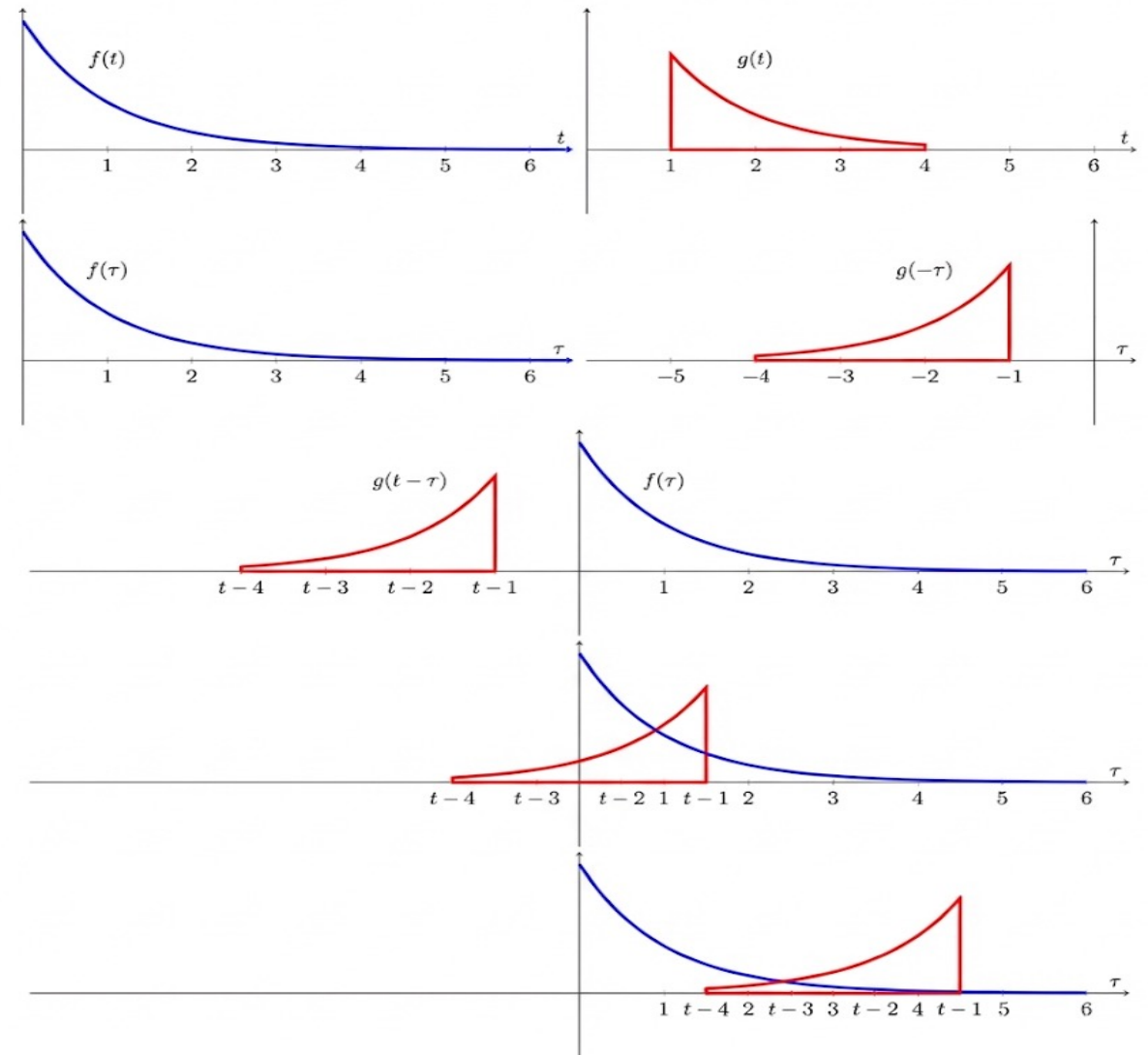
02 April 2026

Recap: Convolution Integral

$$(f * g)(t) := \int_{-\infty}^{\infty} f(\tau)g(t - \tau) d\tau$$

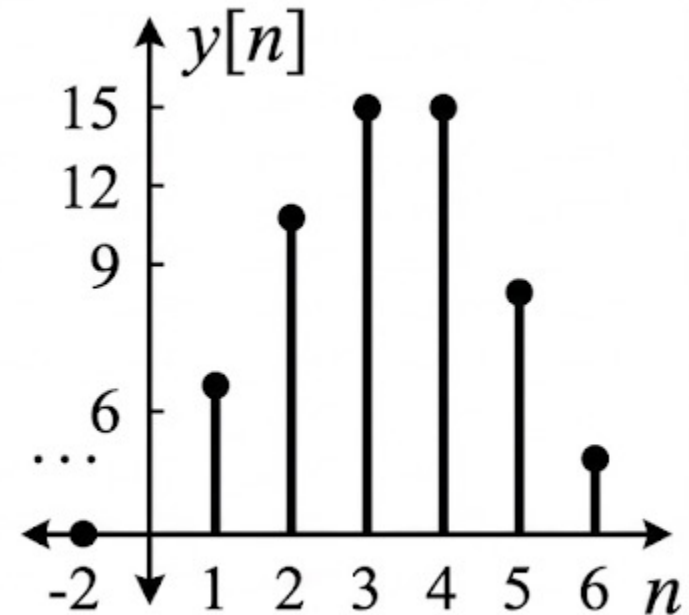
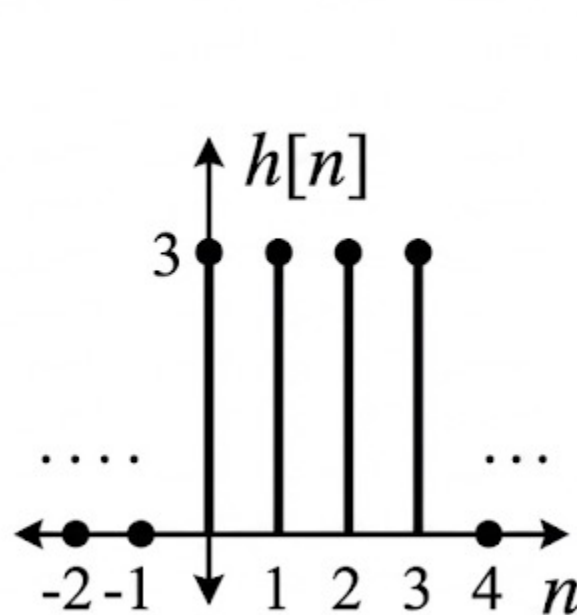
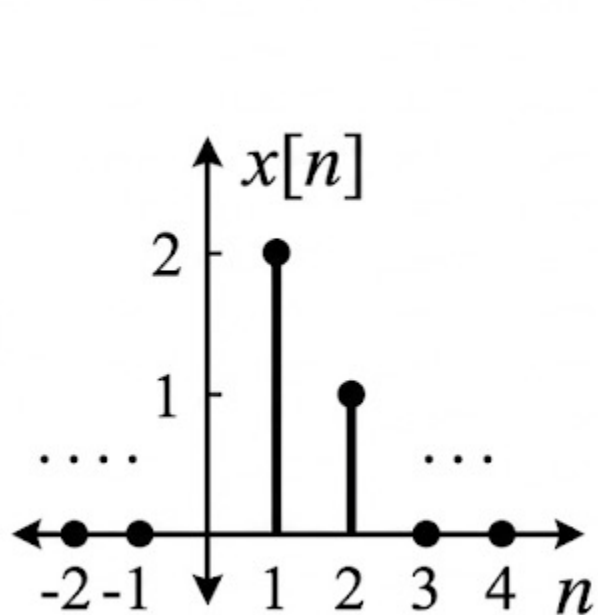
Flip $g(t)$ and slide over τ axis

Compute area under product curve as function of shift t



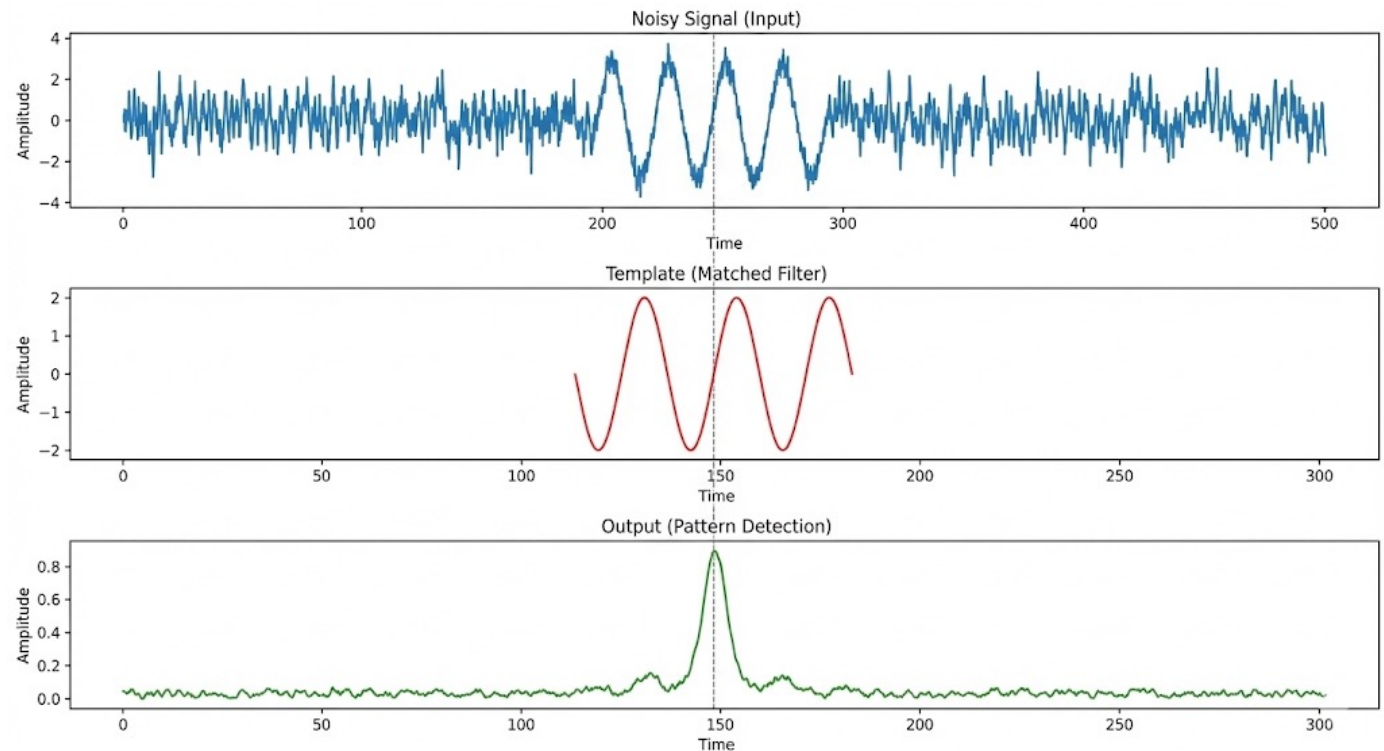
Recap: Discrete Version from Signal Processing

$$y[n] = (x * h)[n] = \sum_{m=-\infty}^{\infty} x[m]h[n-m]$$



“Pattern Detection” with Matched Filter

- Used for pattern recognition
- Convolve with inverted version of desired pattern
- Output will have peak location of pattern
- CNNs actually do correlation, i.e., Correlation slides a filter over the image and computes a **similarity score** at each location.



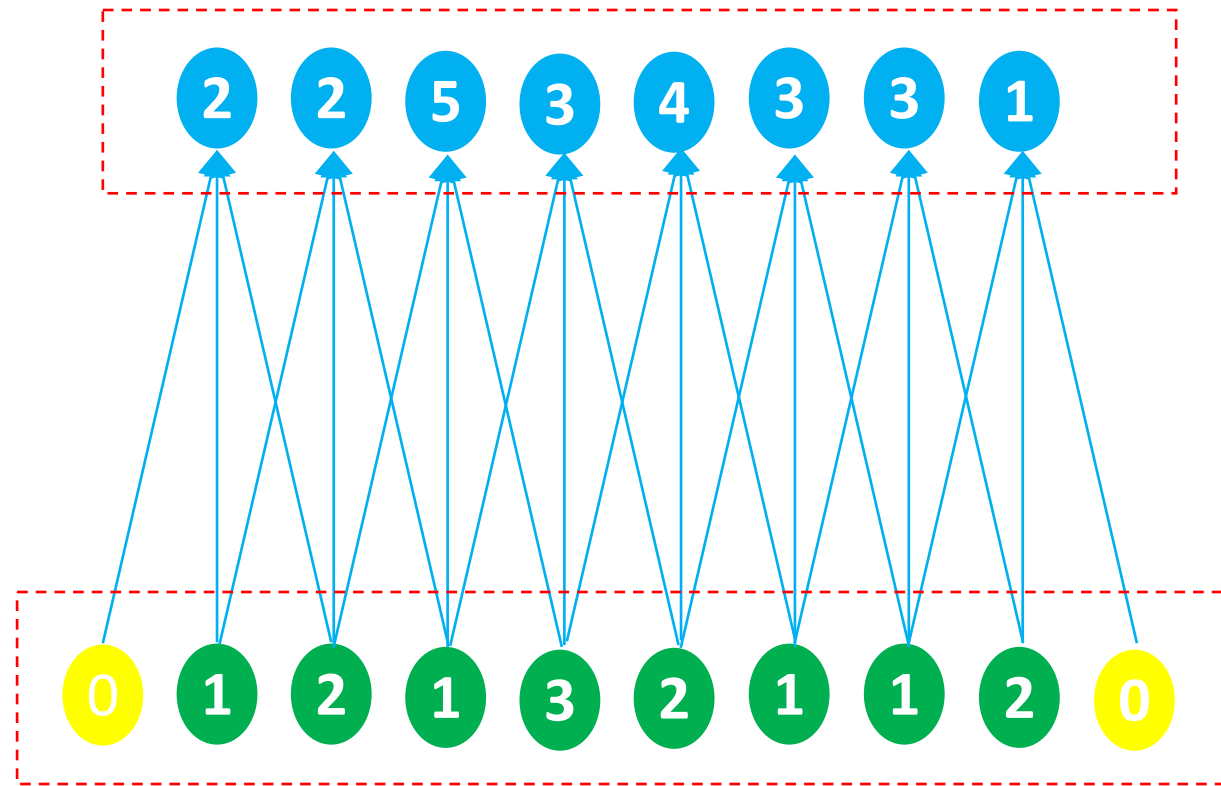
Convolutions with Padding

Filter-1

1

0

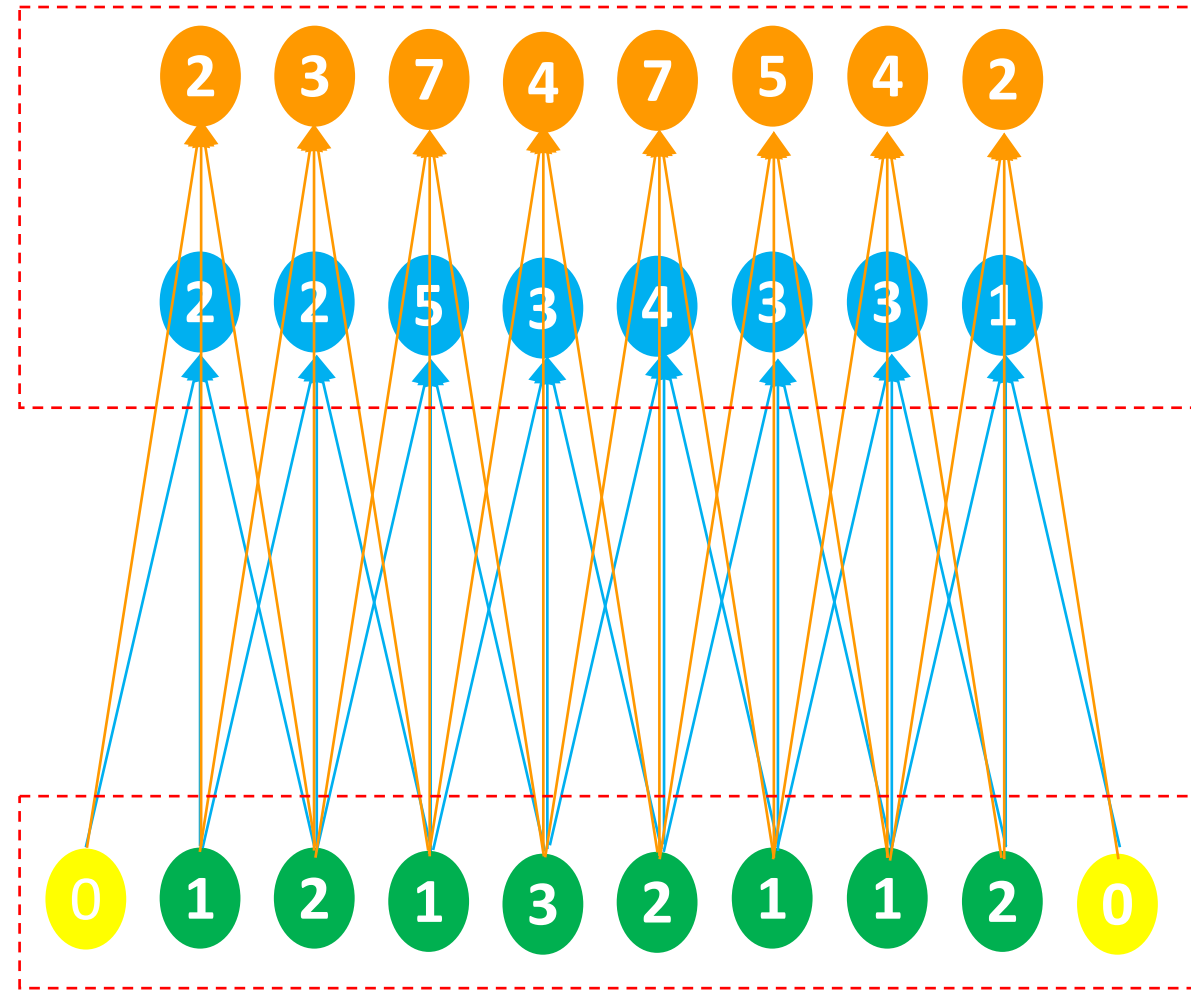
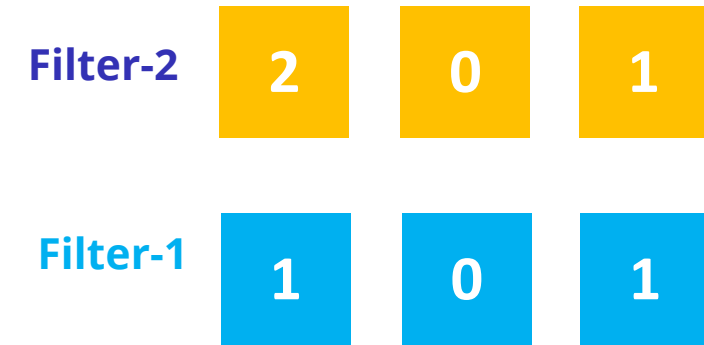
1



Weights that eventually we learn with
Backpropagation (filter size = #
Parameters = # weights = 3)

Pad Extra Samples so that
output size does not reduce

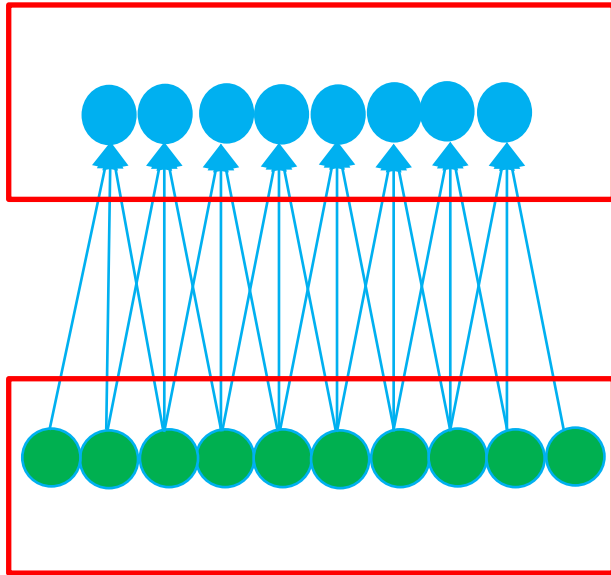
Multiple Convolutional Filters



Weights that eventually we learn with
Backpropagation (filter size = #
Parameters = # weights = 3)

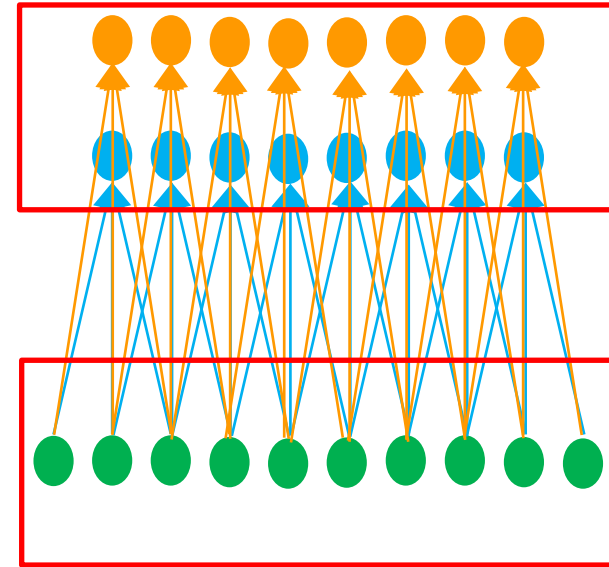
Pad Extra Samples so that
output size does not reduce

Convolution layer: Different Possibilities



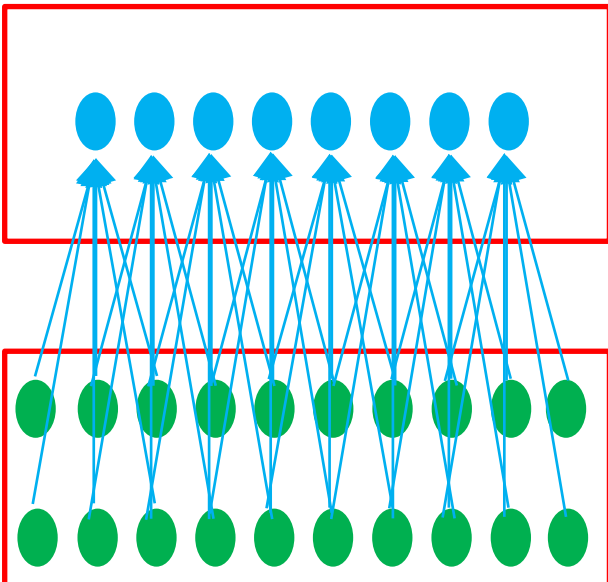
Channels:

- I/P = 1
- O/P = 1
- #Parameters = 3



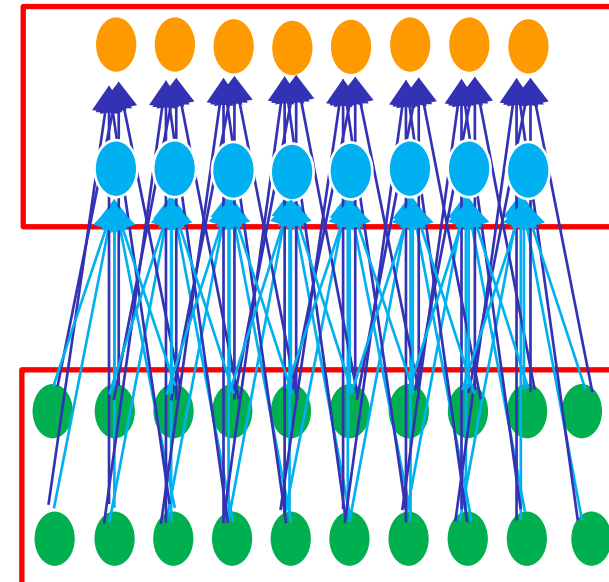
Channels:

- I/P = 1
- O/P = 2
- #Parameters = 6



Channels:

- I/P = 2
- O/P = 1
- #Parameters = 6



Channels:

- I/P = 2
- O/P = 2
- #Parameters = 12

Convolution layer: Different Possibilities



Channels:

- I/P = 1
- O/P = 1
- #Parameters = 3



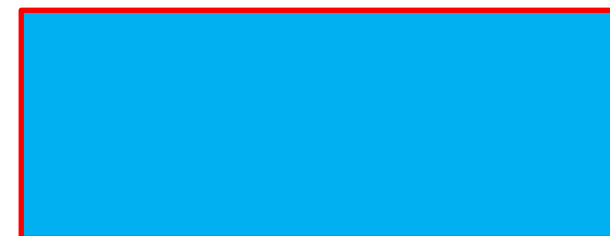
Channels:

- I/P = 1
- O/P = 2
- #Parameters = 6



Channels:

- I/P = 2
- O/P = 1
- #Parameters = 6



Channels:

- I/P = 2
- O/P = 2
- #Parameters = 12

2D Convolution: Definition

$$y[m, n] = x[m, n] * h[m, n] = \sum_{j=-\infty}^{\infty} \sum_{i=-\infty}^{\infty} x[i, j] \cdot h[m - i, n - j]$$

1	2	1		
0	0	1	2	3
-1	-2	4	5	6
		7	8	9

$y[0, 0]$

1	2	1		
0	0	2	3	
-1	-2	5	6	
		7	8	9

$y[1, 0]$

	1	2	1	
	0	2	3	0
1	4	5	6	-1
		7	8	9

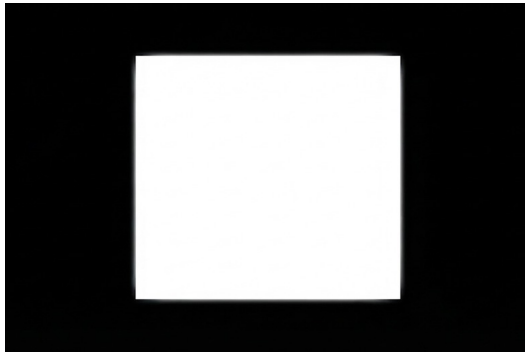
$y[2, 0]$

		1	2	1	
1	2	1			3
0	0	4	5	6	
-1	-2	7	8	9	

$y[0, 1]$

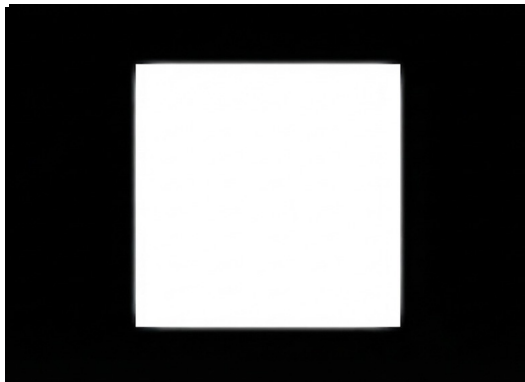
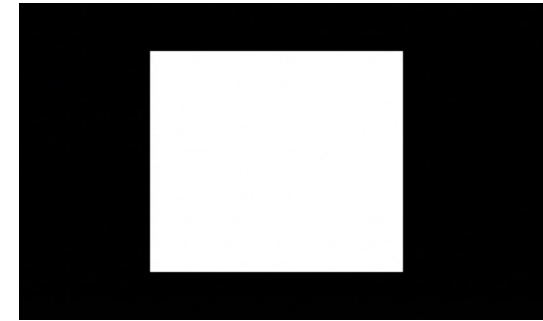
and so on...

Examples of Kernels from Image Processing



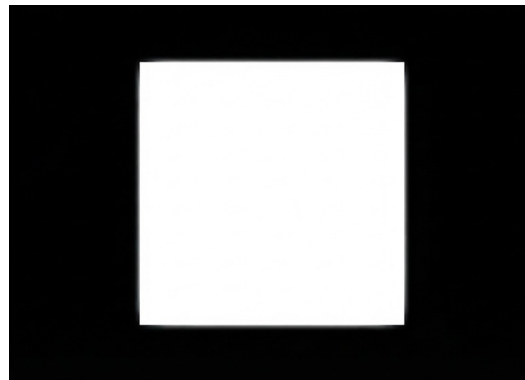
$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Sharpening



$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Blur



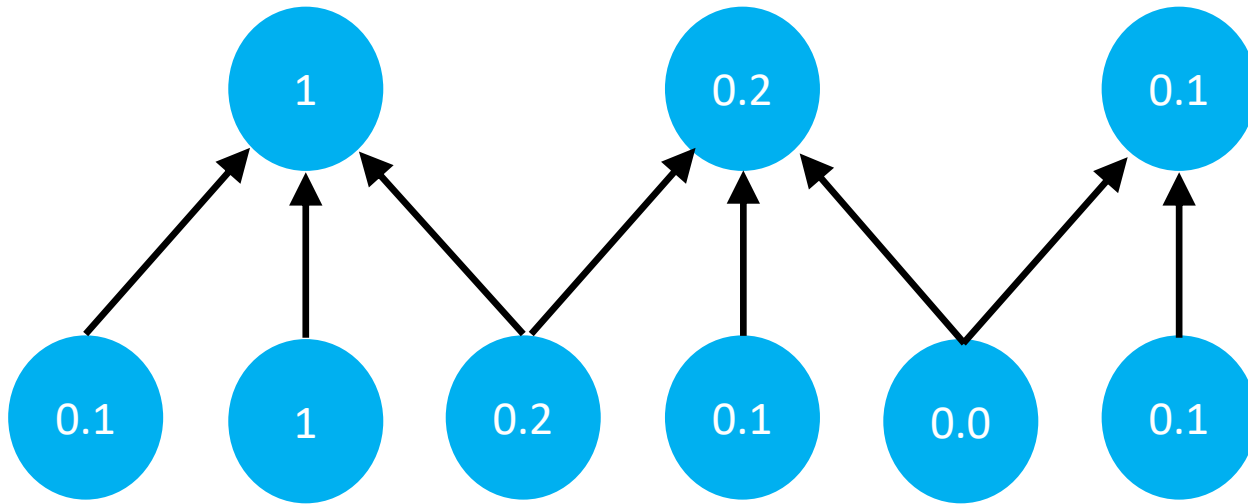
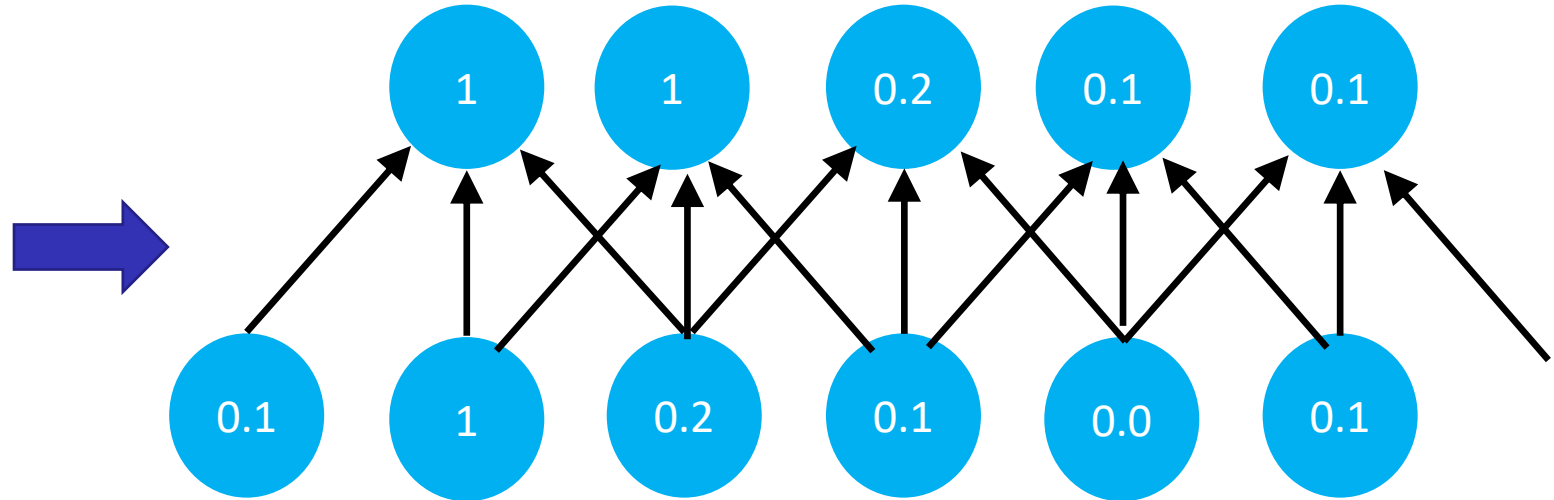
$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Gaussian blur



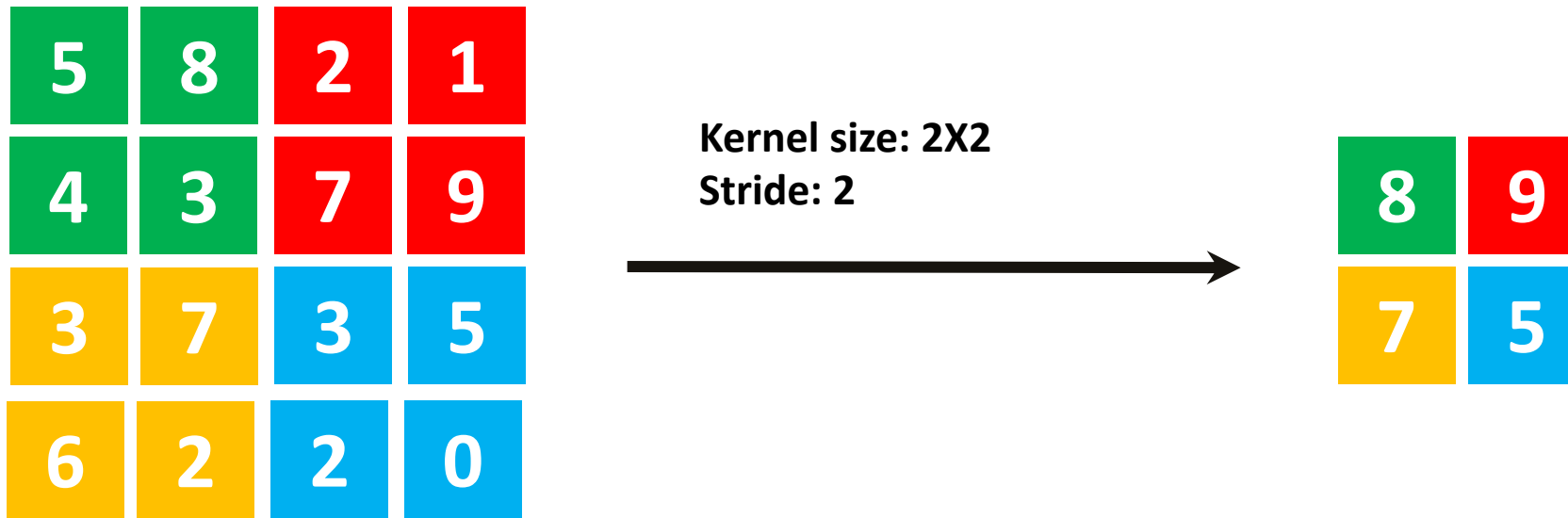
Max Pool and Stride

- Window Size = 3
- Stride = 1



- Window Size = 3
- Stride = 2

Max pooling in 2-D



Convolution

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

These are the network parameters to be learned.

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

⋮ ⋮

Each filter detects a small pattern (3 x 3).

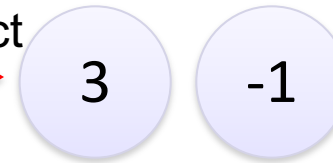
Convolution

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

Dot
product



1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

Convolution

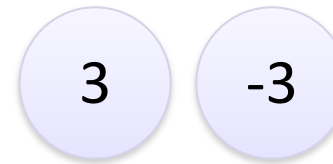
If stride=2

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1



Convolution

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

Convolution

stride=1

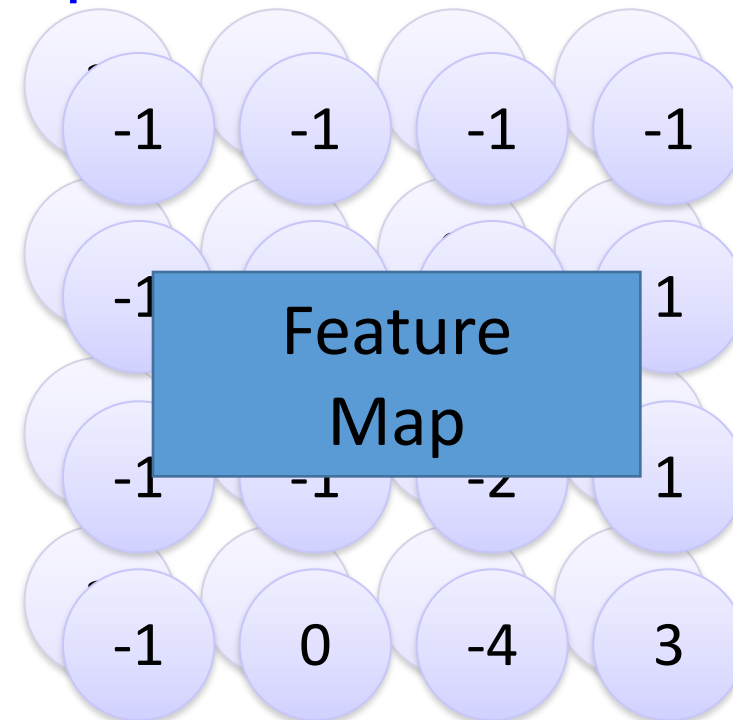
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

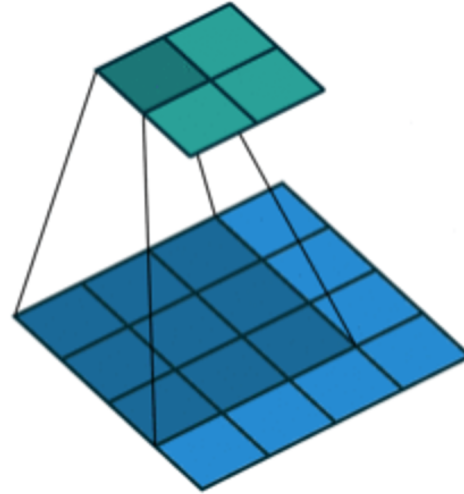
Repeat this for each filter



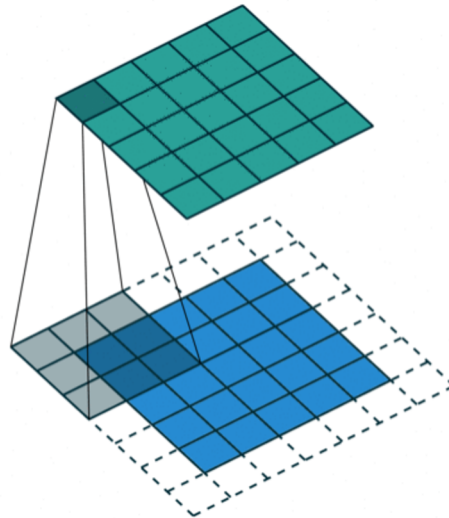
Two 4 x 4 images
Forming 2 x 4 x 4 matrix

CNNs

- Window size
- Stride
- Padding



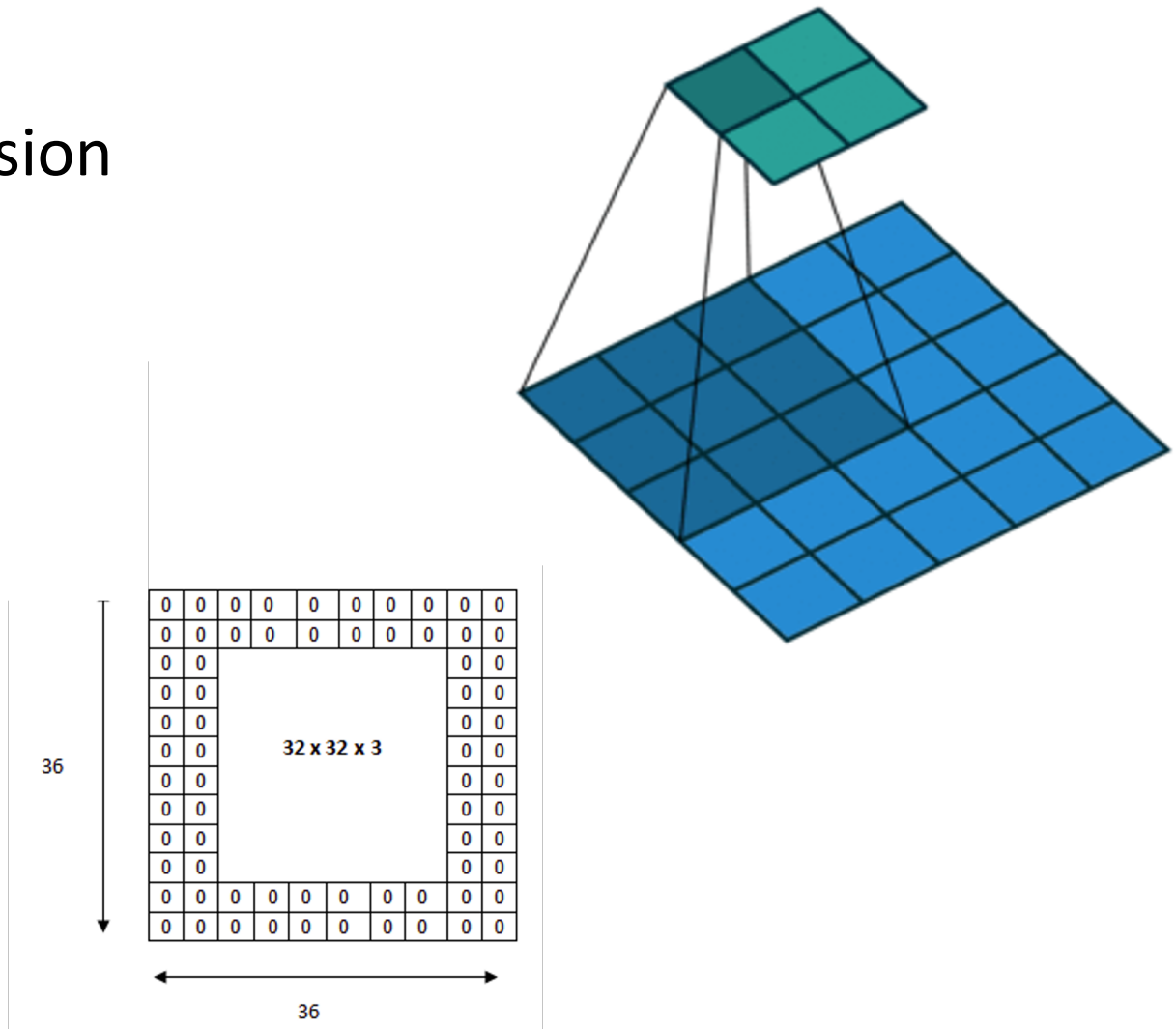
Window size: 3x3
Stride: 1
Padding: 0



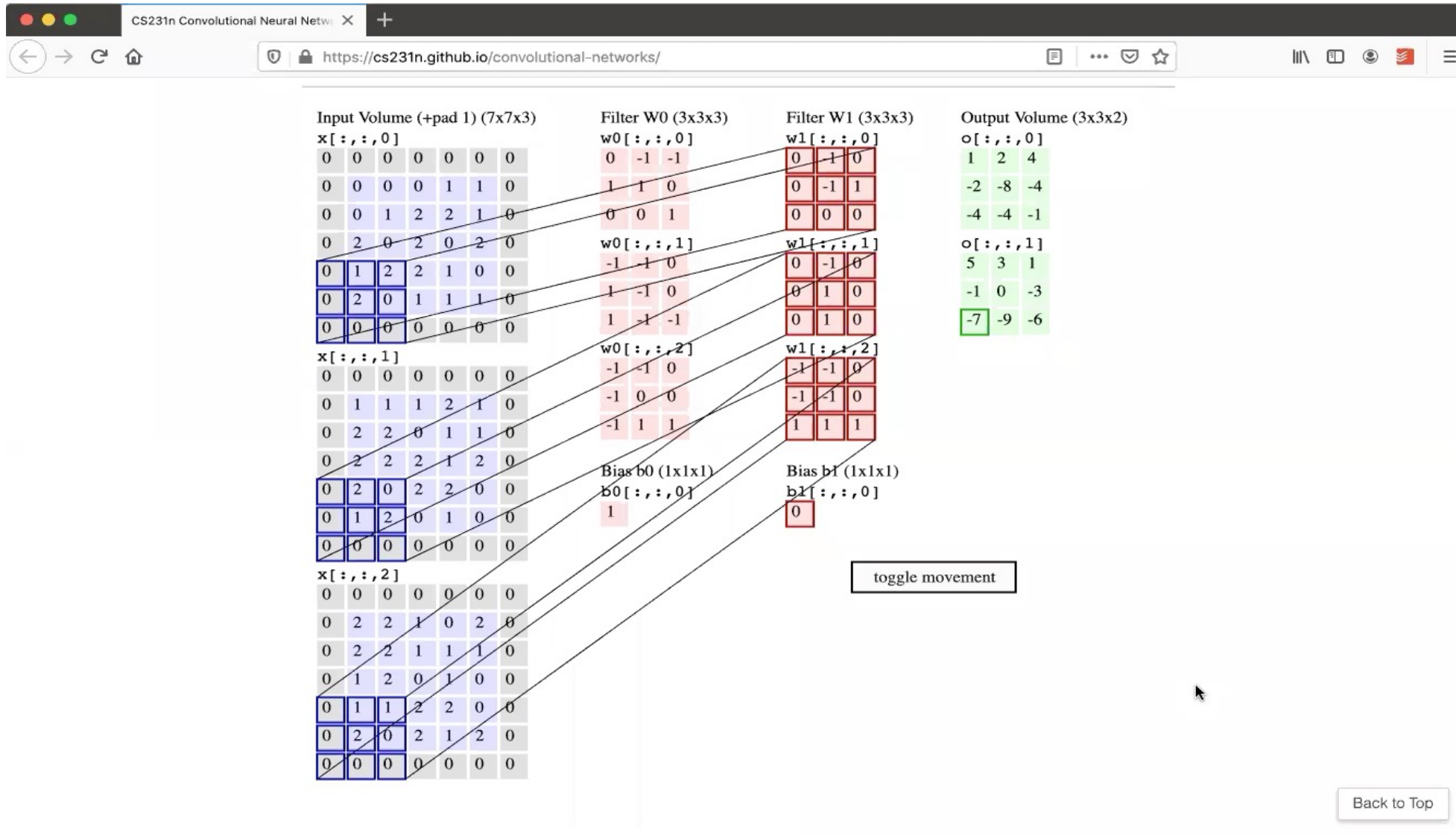
Window size: 3x3
Stride: 1
Padding: 1

- Strides reduces dimension

$$O = \frac{(W - K + 2P)}{S} + 1$$



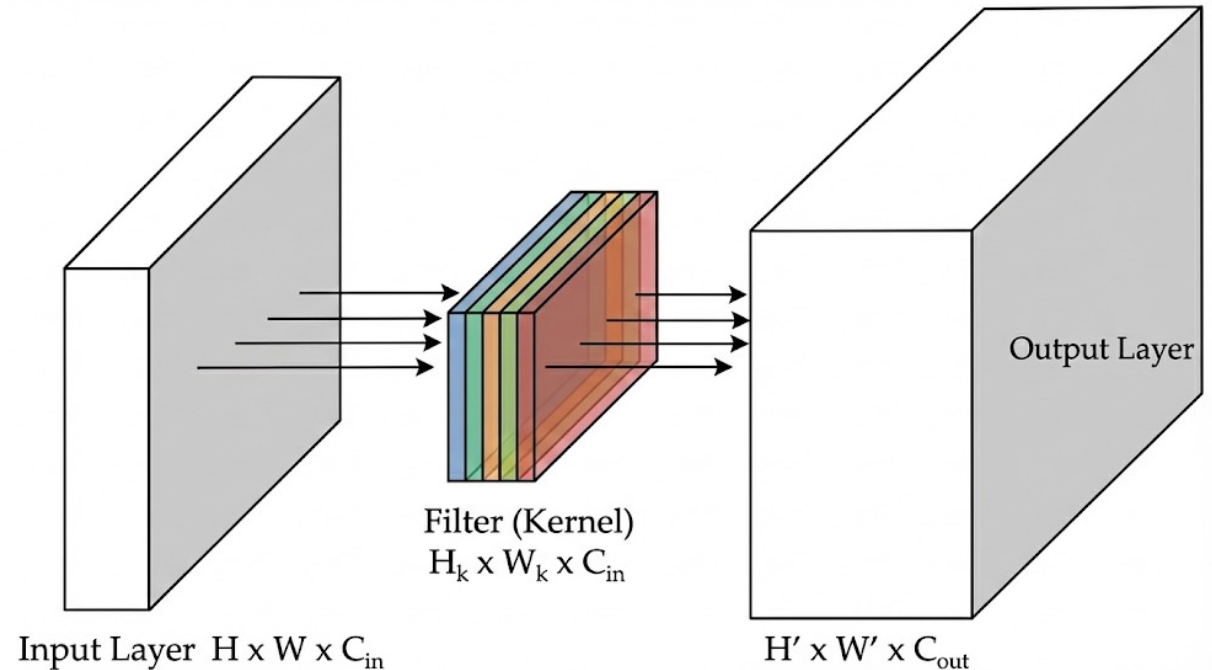
Input, Output Channels : Multiple Filters



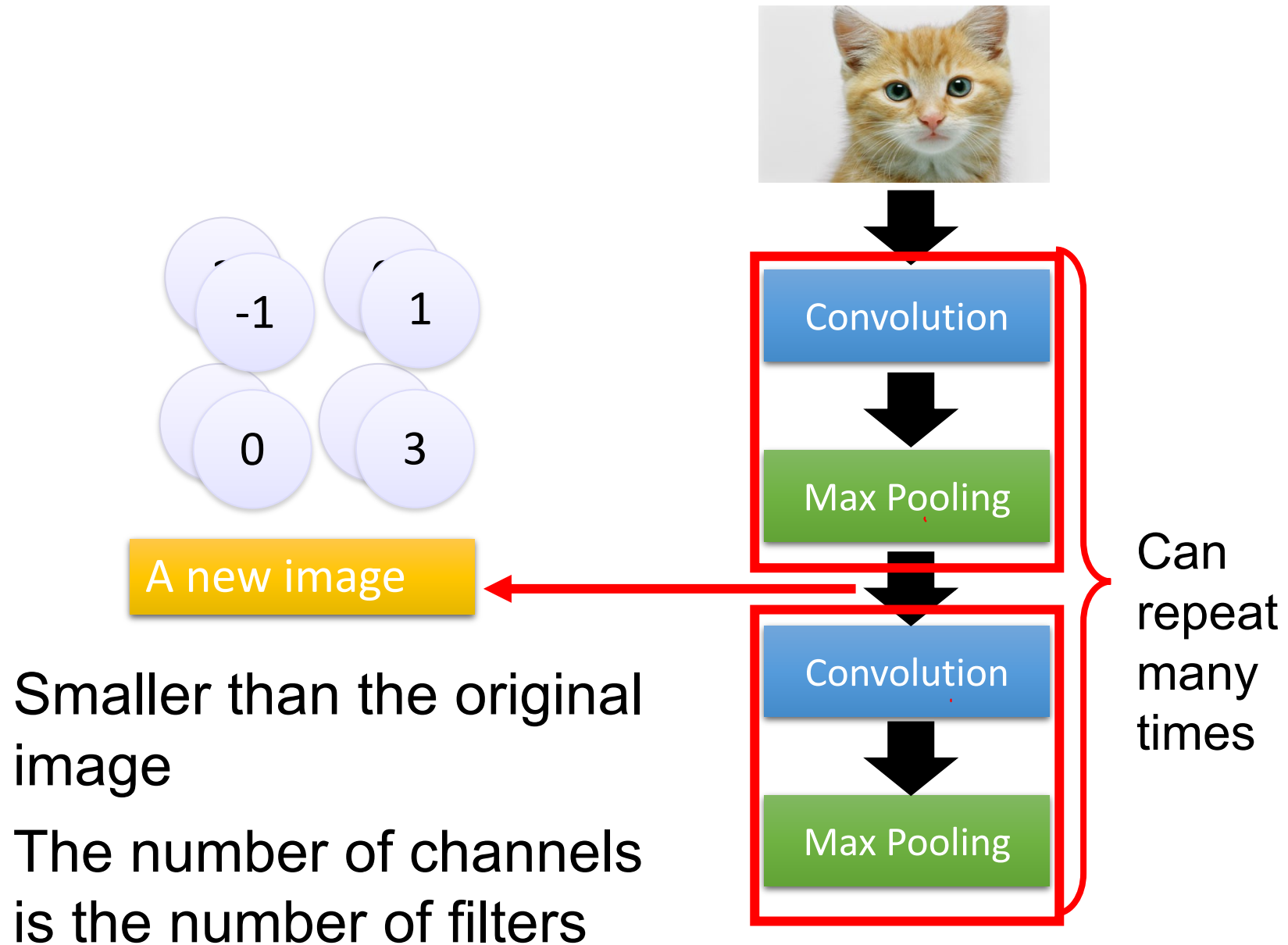
Input-Output Tensors

A $H \times W \times C_{in}$ input on convolution with a kernel of size $H_k \times W_k \times C_{in}$ gives a single-channel output of dimension $H' \times W'$

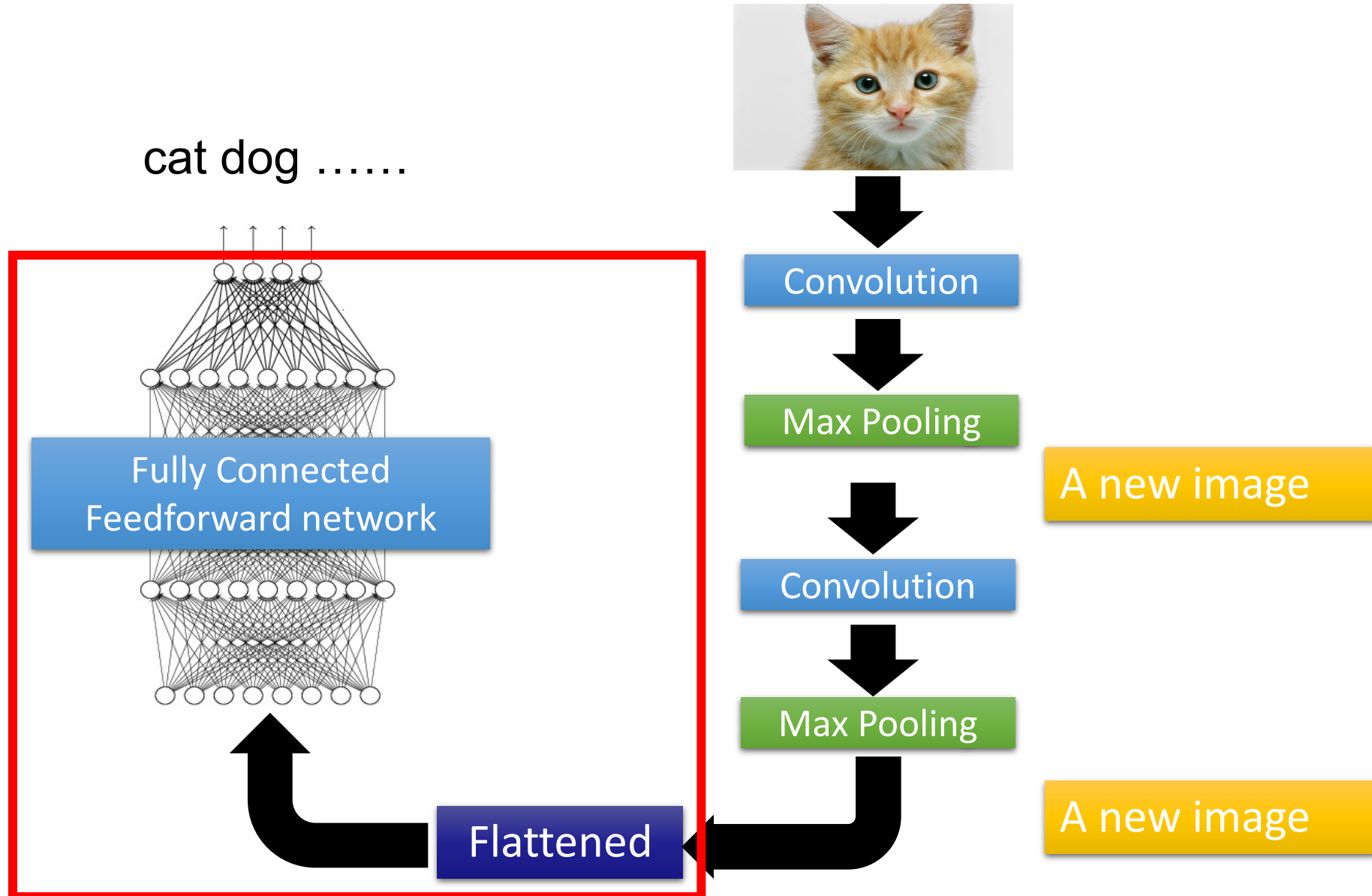
To get C_{out} channels, we need C_{out} such filters



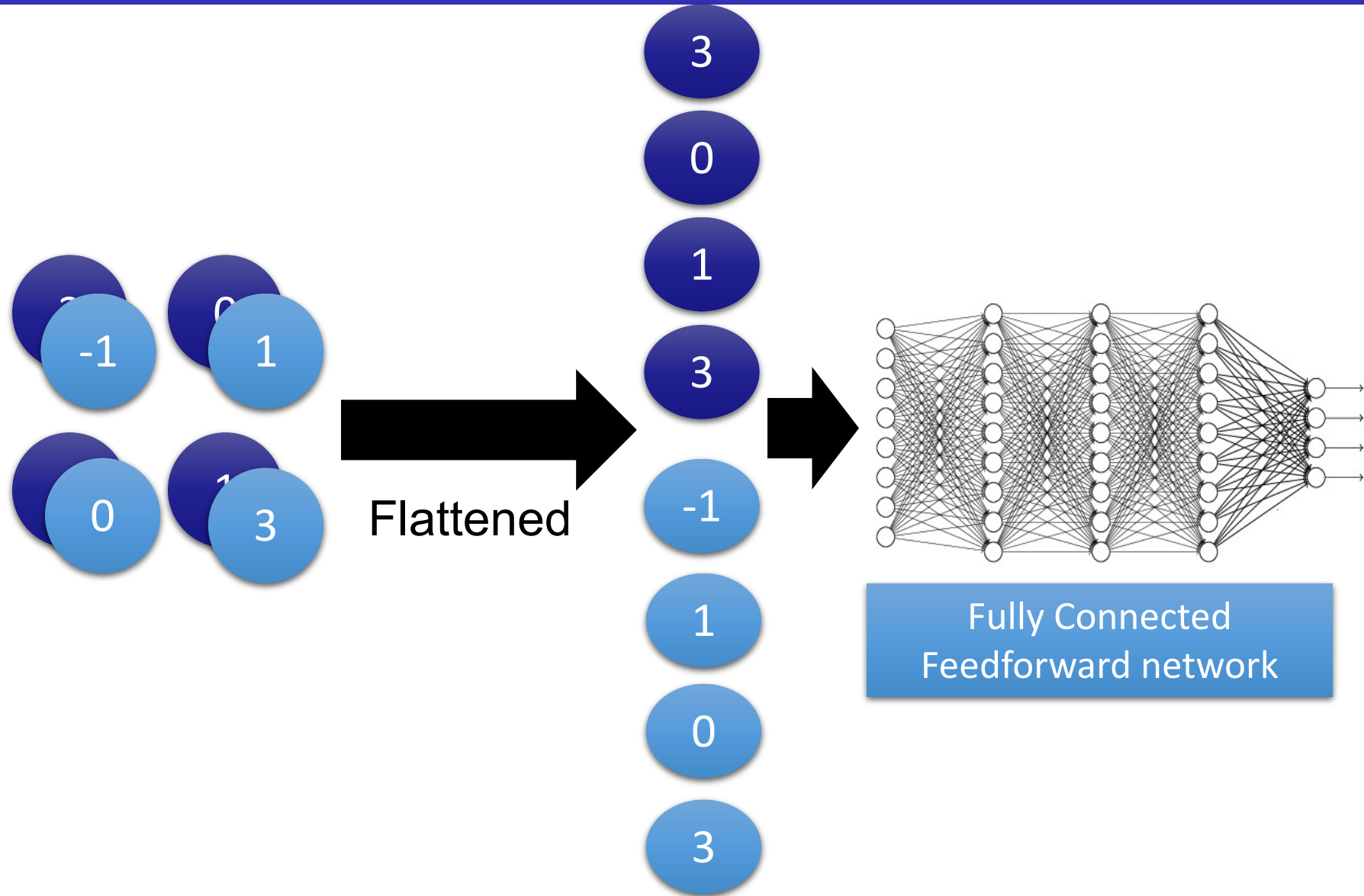
The Whole CNN



The whole CNN

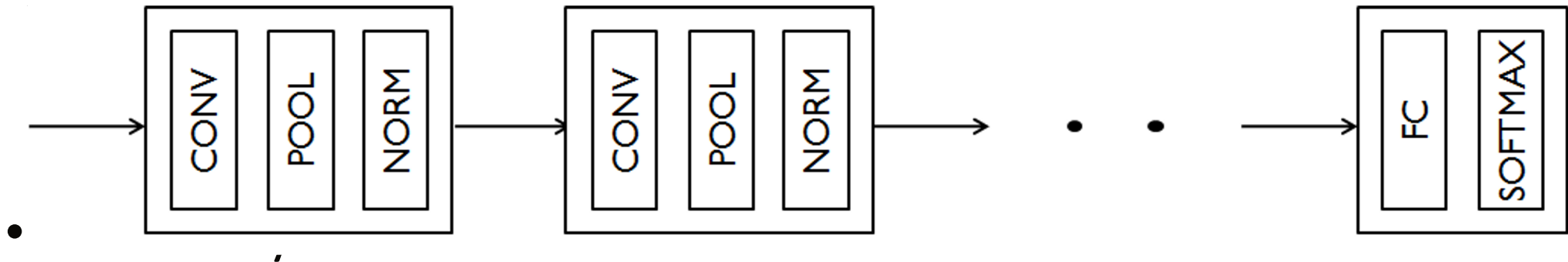


Flattening



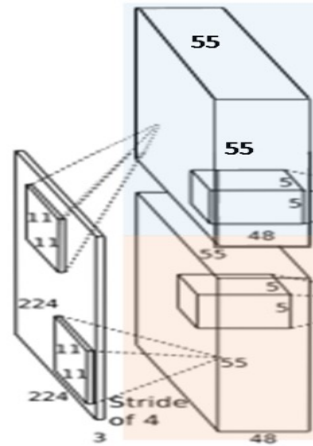
Typical Architecture

- A typical deep convolutional network



- Pooling
- Normalization
- Fully connected
- etc.

Parameter Calculation

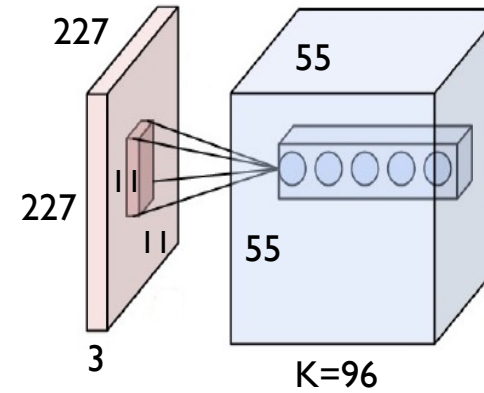


- Filter Size F
 - Input volume streams be D
 - # filters be K
 - # parameters in a layer is $(F \cdot F \cdot D) \cdot K$
- } Hyper parameters

Example:

For layer 1, Input images are $227 \times 227 \times 3$

- $F = 11$ and $K = 96$
- Each filter has $11 \times 11 \times 3 = 363$ and 1 (bias) i.e., 364 weights
- # weights = $364 \times 96 = 35 \text{ K}$ (approx.)



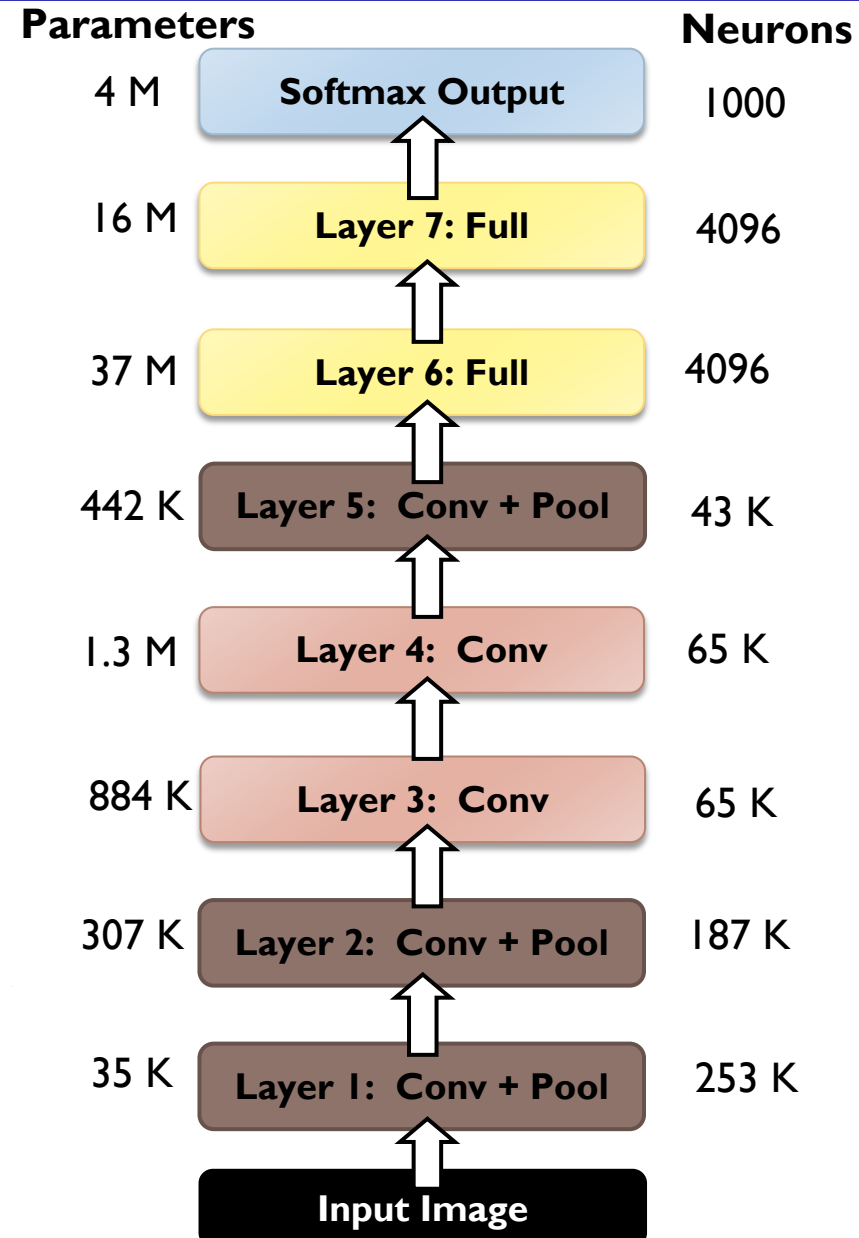
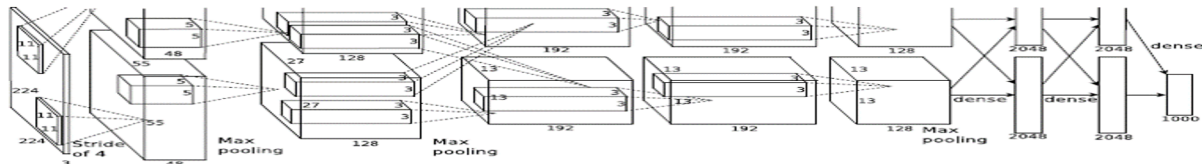
- Stride S
 - Zero padding P
 - Input Size: $W1 \times H1 \times D1$
 - Output Size: $W2 \times H2 \times D2$
 - $W2 = [(W1 - F + 2P) / S] + 1$ and $D2 = K$
- } Hyper parameters

- $S = 4, W1 = 227, F = 11, P = 0, D2 = 96$
- $W2 = (227 - 11) / 4 + 1 = 55$
- Output Size: **$55 \times 55 \times 96$**

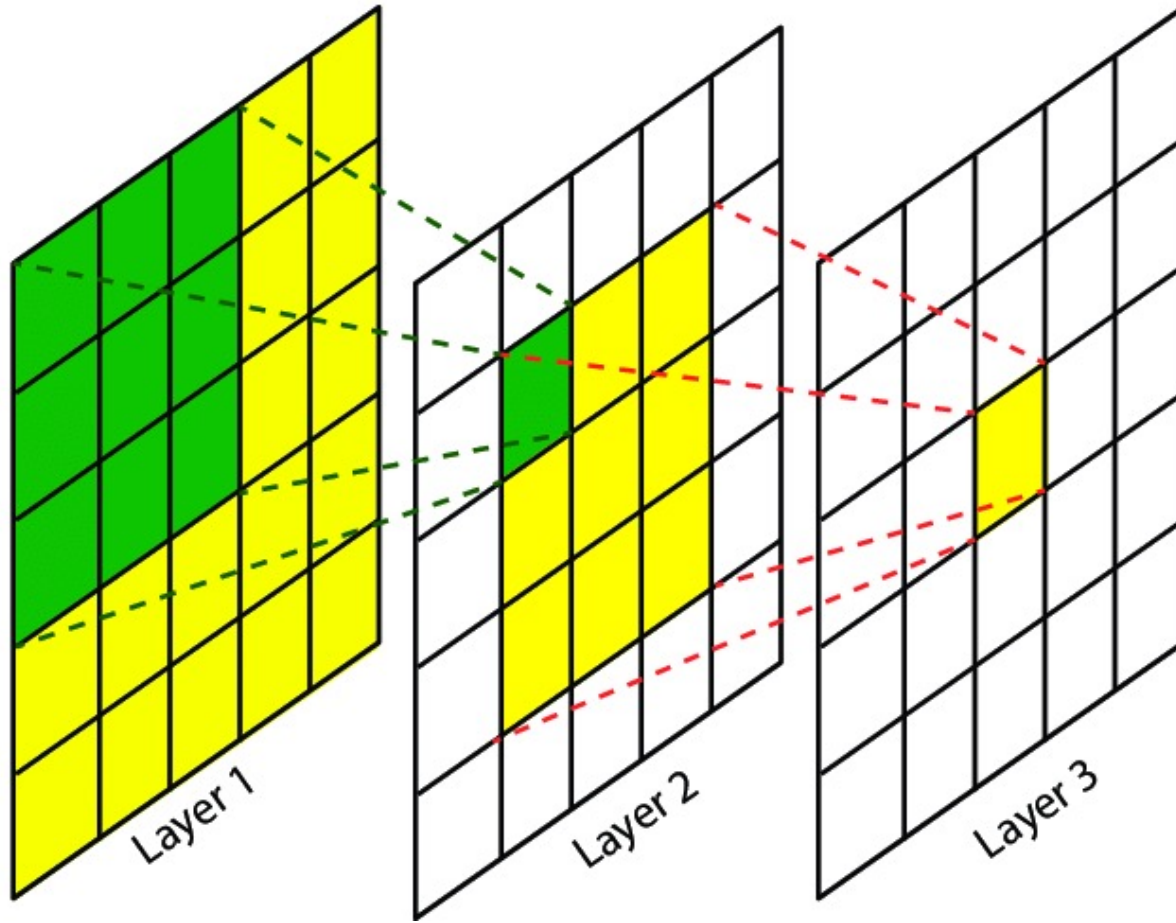
AlexNet Architecture

Trained with **stochastic gradient descent**

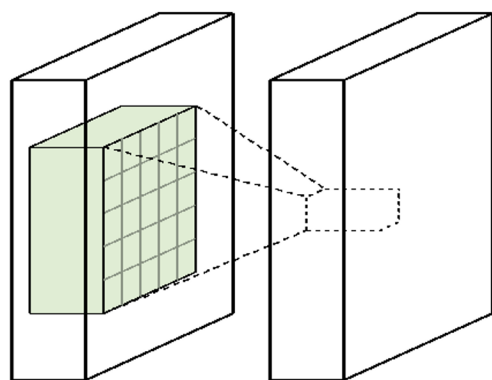
- on two NVIDIA GTX 580 3GB GPUs
- for about a week
- 650,000 neurons
- 60 M parameters
- 630 M connections
- Final feature layer: 4096-dimensional



Replacing 5X5 by two 3X3



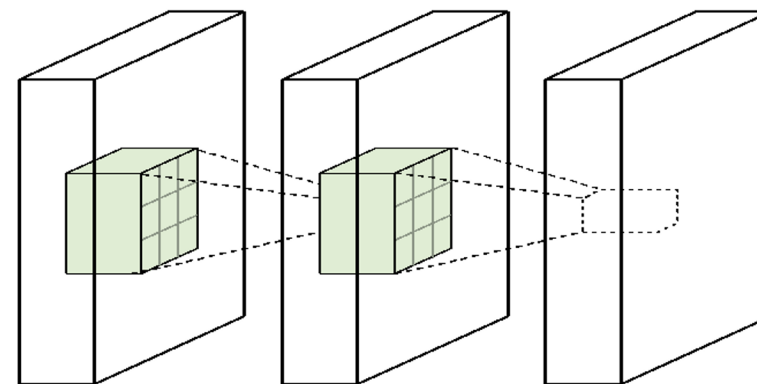
Design Guidelines



5 × 5 filters
+ ReLU

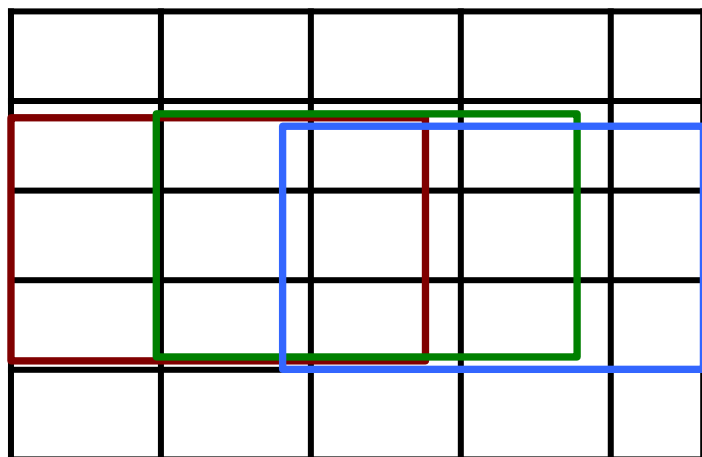


5 X 5 C > 2 X 3 X 3 C

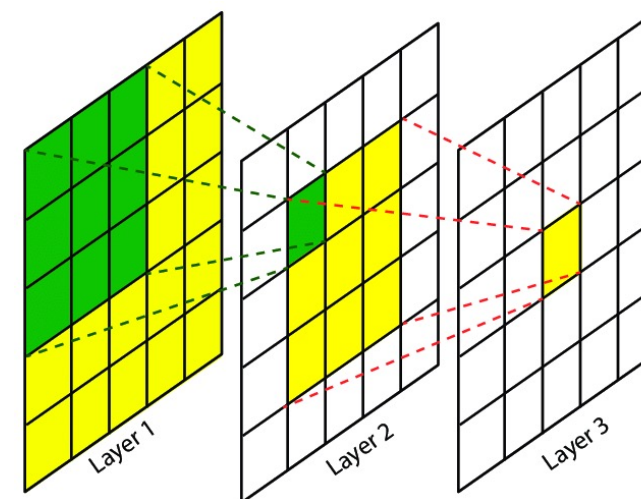


3 × 3 filters
+ ReLU

3 × 3 filters
+ ReLU



1. Less Parameters; Faster
2. Same Receptive Field
3. More nonlinearities (2 ReLU)



Improvements

Improvements over AlexNet - I

- Smaller receptive field throughout the network
 - AlexNet used $k = 11 \times 11$ and $s = 4$
 - VGGNet used $k = 3 \times 3$ and $s = 1$

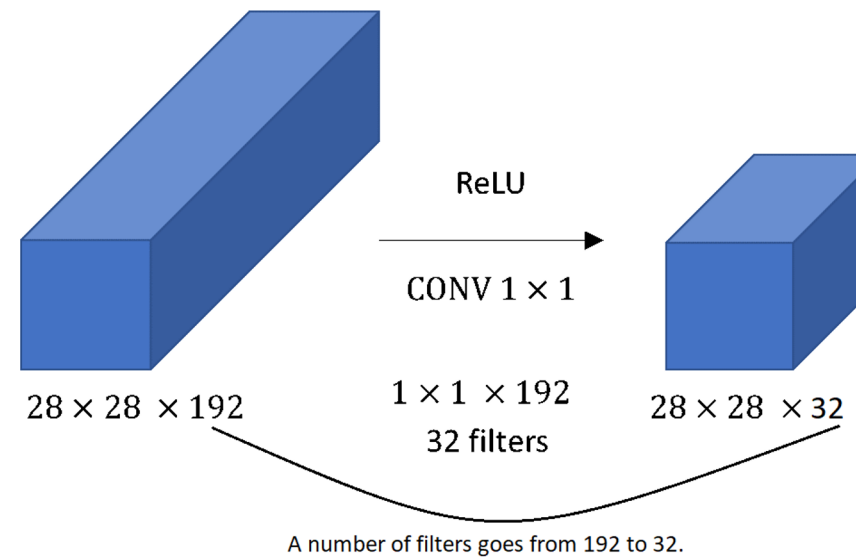
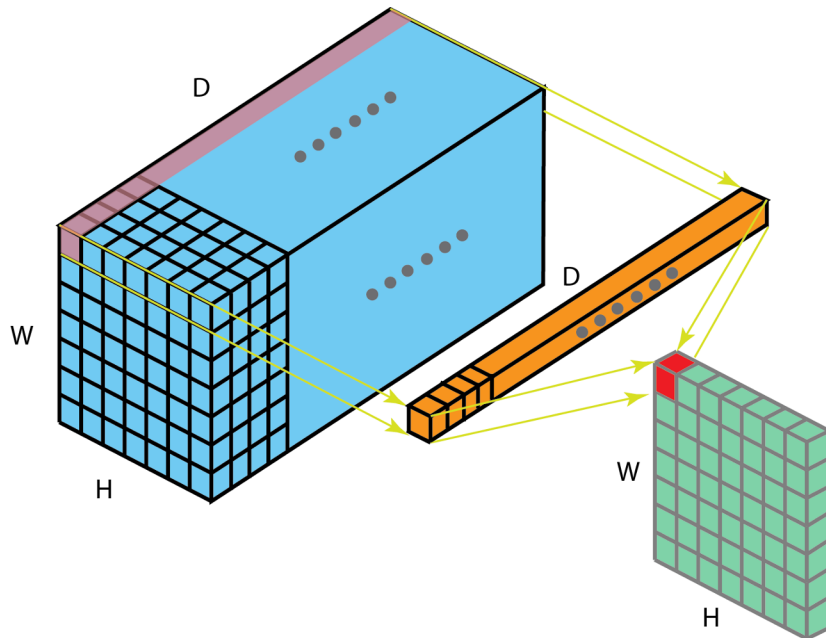
Intuition: A stack of **two** 3×3 convolutional layer is equivalent to a 5×5 convolution layer

- More non-linearity
- Less number of parameters

1 X 1 Convolution

Improvements over AlexNet - II

- 1×1 convolution
 - Increases the nonlinearity without affecting the receptive field

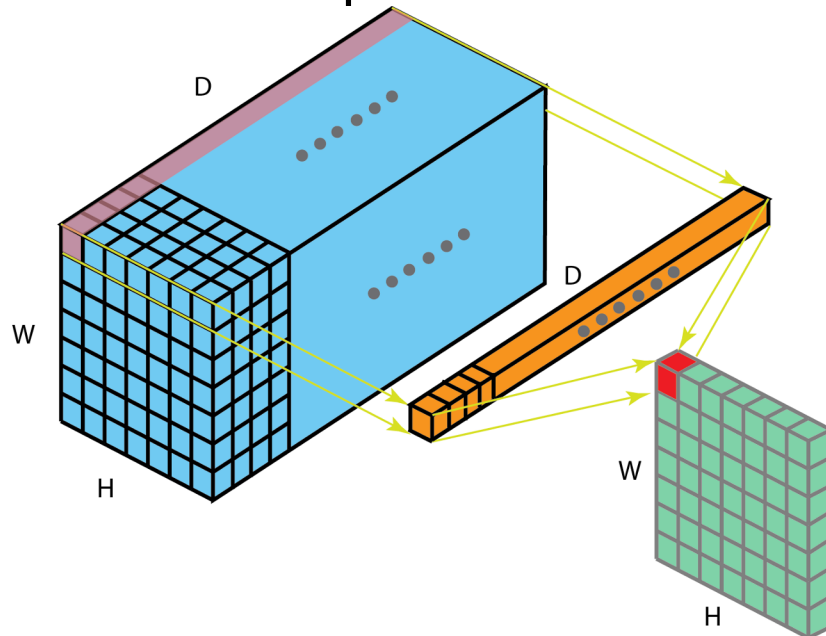


1 X 1 Convolution

Improvements over AlexNet - II

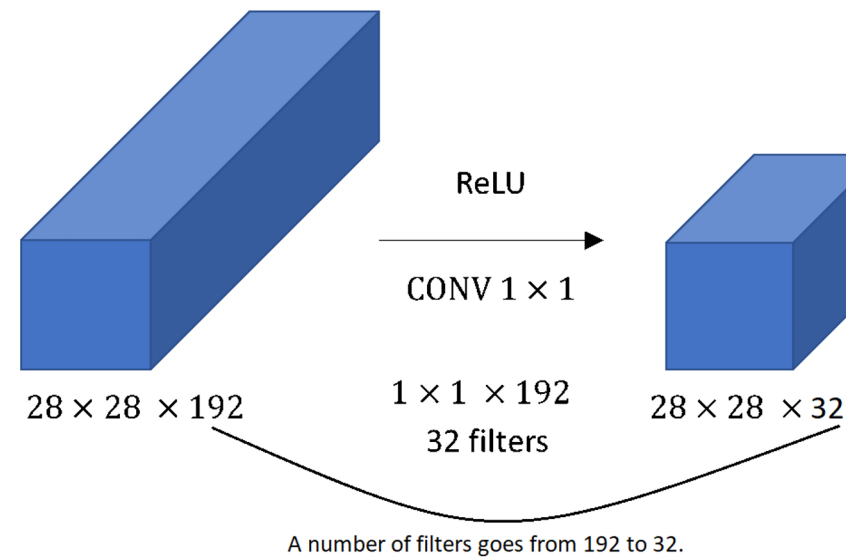
- 1×1 convolution

Increases the non-linearity without affecting the receptive field

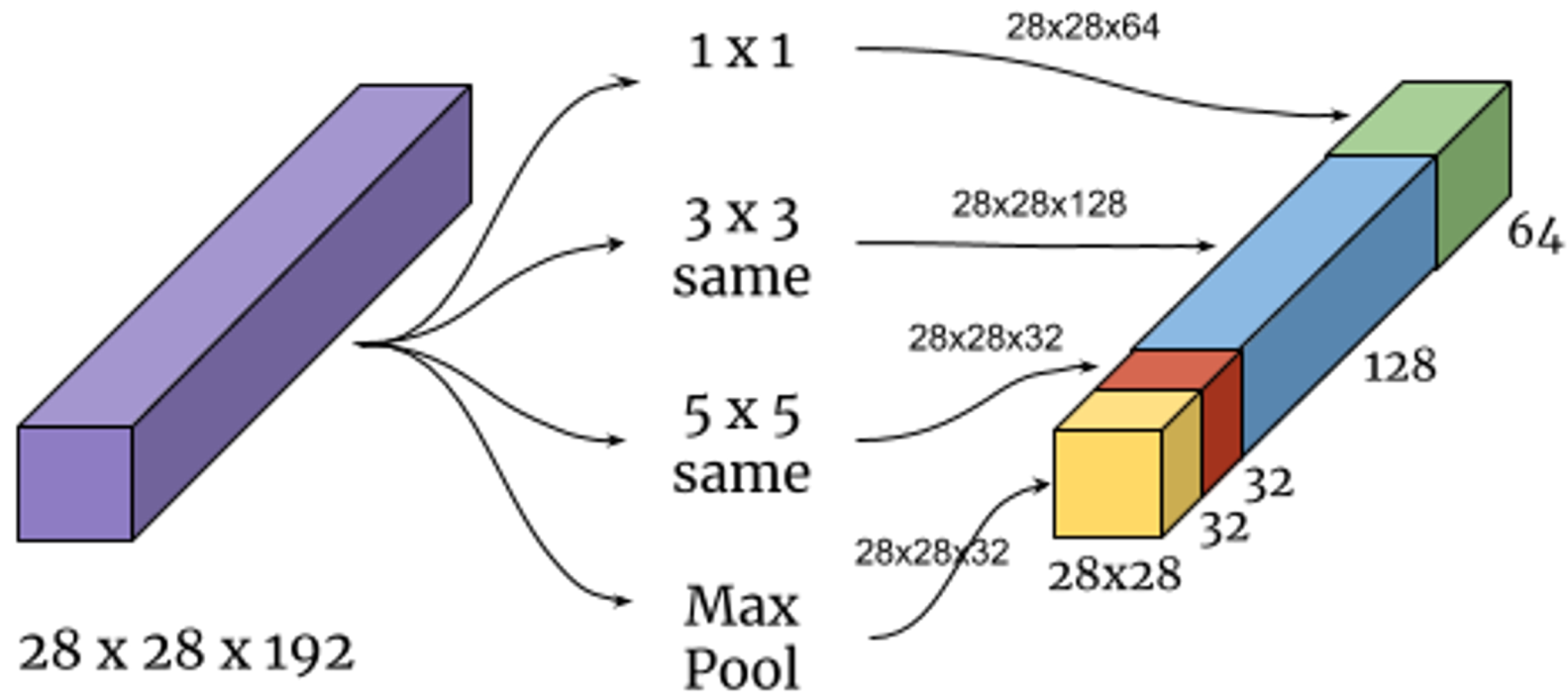


Usage:

- Dimensionality reduction
- Building deeper network w/ large increase in parameters
- Increased non-linearity



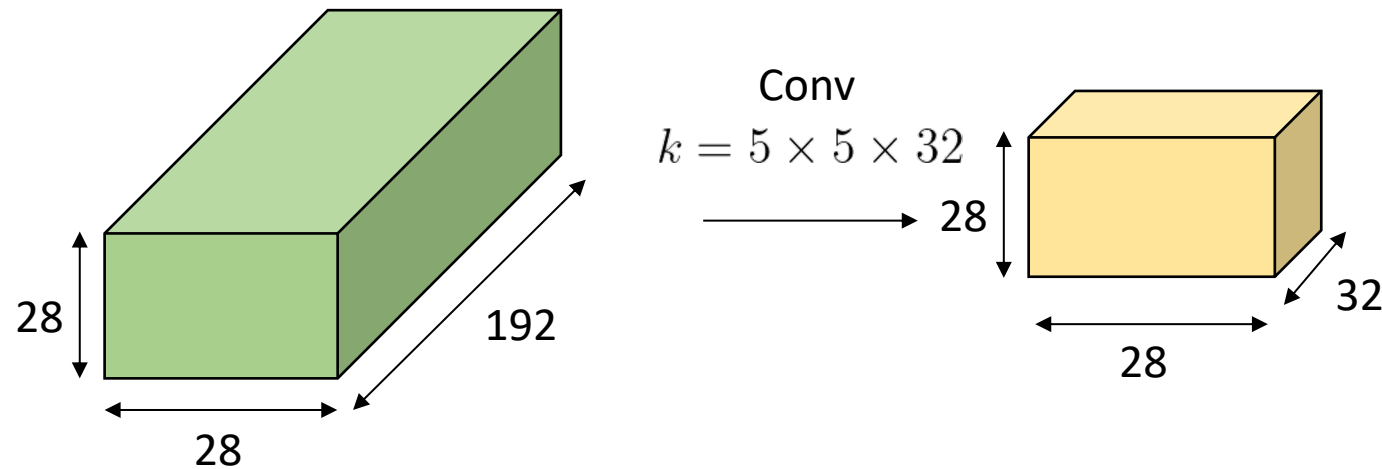
Use multiple kernels/windows



This would take a large number of computations! Can we reduce it?

Role of Bottleneck

Motivation of using a 1x1 Convolutional Layer:

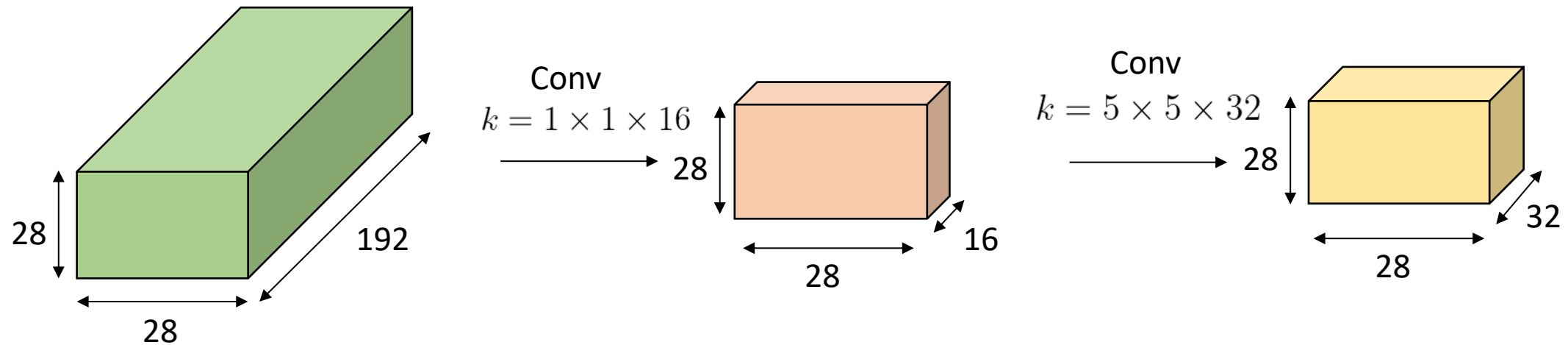


Total number of operations:

$$(28 \times 28 \times 32) \times (5 \times 5 \times 192) = 120\text{M}$$

Role of Bottleneck

Motivation:



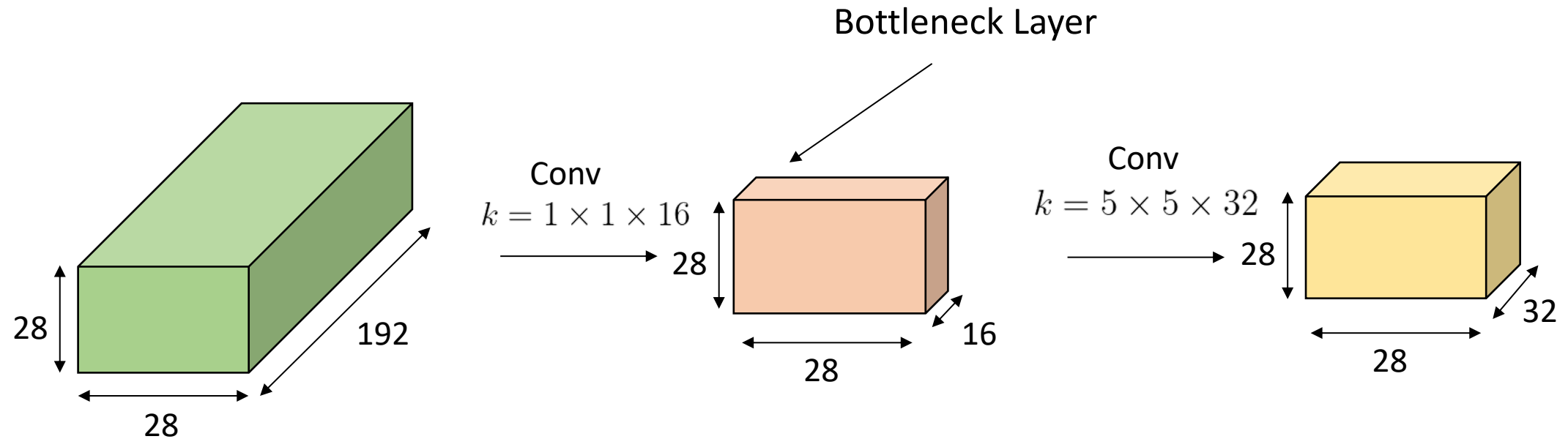
Total number of operations:

$$(28 \times 28 \times 16) \times (1 \times 1 \times 192) + (28 \times 28 \times 32) \times (5 \times 5 \times 16) = 12.4\text{M}$$

Role of Bottleneck



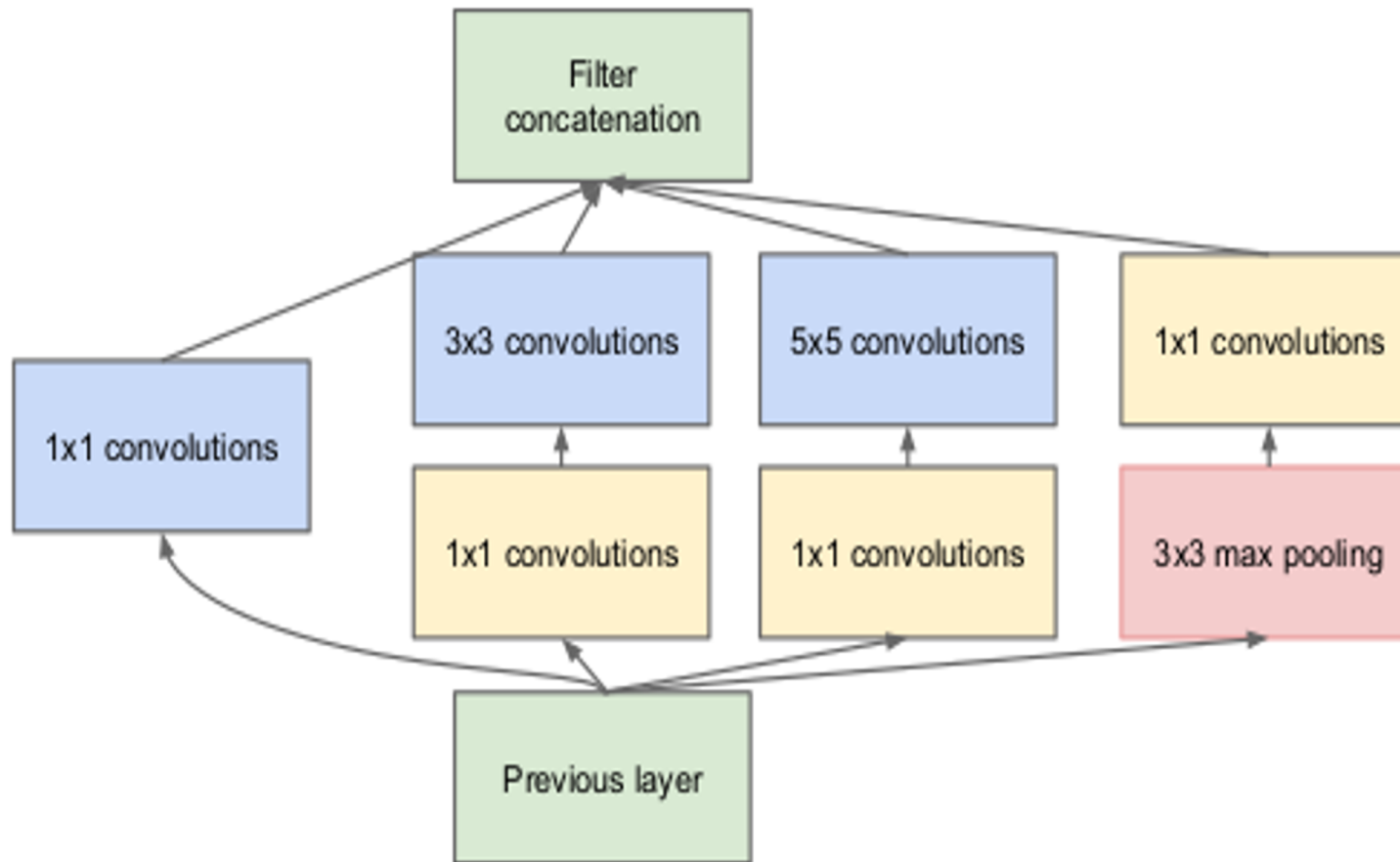
Motivation:



Total number of operations:

$$(28 \times 28 \times 16) \times (1 \times 1 \times 192) + (28 \times 28 \times 32) \times (5 \times 5 \times 16) = 12.4\text{M}$$

The Inception Layer (in googlenet)



Questions?